



Université Joseph Ki-ZERBO



Unité de formation et de
recherche en
sciences exactes et
appliquées (UFR/SEA)

RAPPORT DE PROJET

Conception et utilisation d'un service REST

Tronc commun Master 1 Informatique

Membres du groupe 12 :

OUATTARA Barkissa

KABORE Paulin

Enseignant :

Dr SOMDA

Année académique : 2025/2026

Introduction

Dans le développement des applications modernes, les services web occupent une place essentielle. Ils permettent à différentes applications de communiquer entre elles de manière standardisée, généralement via le protocole HTTP.

Parmi les styles d'architecture les plus utilisés, REST (Representational State Transfer) s'impose aujourd'hui comme une référence. Il repose sur des principes simples tels que l'utilisation des méthodes HTTP, l'identification des ressources par des URI et l'échange de données au format JSON.

Dans ce travail pratique, nous avons mis en place une API REST permettant de gérer des cours universitaires. L'objectif principal est de comprendre comment structurer une API, manipuler les différentes opérations CRUD et interpréter les réponses HTTP.

Chapitre 1

Contexte métier

Le sujet propose de développer une API REST de gestion de cours universitaires. La ressource principale manipulée dans l'application est **Cours**.

Listing 1.1 – Ressource principale : Cours

```
1 {  
2   "id": 1,  
3   "titre": "Services Web",  
4   "enseignant": "Dr. Somda",  
5   "volumeHoraire": 30,  
6   "actif": true  
7 }
```

Cette ressource représente un cours universitaire caractérisé par :

- un identifiant unique ;
- un titre ;
- le nom de l'enseignant ;
- un volume horaire ;
- un état actif ou inactif.

Chapitre 2

Partie 1 – Modélisation REST

Le sujet demande, dans une première partie, d'identifier les ressources, de définir les URI REST et d'associer les verbes HTTP adaptés.

2.1 Identification des ressources

La ressource principale du projet est **Cours**. Dans une architecture REST, les ressources sont représentées par des noms, et non par des verbes. Ici, la collection de cours est donc naturellement représentée par l'URI `/api/cours`.

2.2 Définition des URI REST

On distingue :

- l'URI de collection pour manipuler l'ensemble des cours ;
- l'URI d'instance pour manipuler un cours particulier identifié par son `id`.

2.3 Association des verbes HTTP

2.4 Justification des choix REST

Le choix des URI et des méthodes repose sur les principes REST :

- `/api/cours` désigne la collection des cours ;
- `/api/cours/{id}` désigne un cours précis ;
- **GET** est utilisé pour consulter des ressources ;
- **POST** permet de créer une nouvelle ressource dans la collection ;
- **PUT** remplace complètement une ressource existante ;
- **PATCH** modifie seulement une partie de la ressource ;
- **DELETE** supprime une ressource.

TABLE 2.1 – Correspondance Actions – Méthodes HTTP – URIs

Action	Méthode	URI	Description
Lister les cours	GET	/api/cours	Retourne tous les cours (tableau JSON)
Lire un cours	GET	/api/cours/{id}	Retourne un cours identifié par id
Créer un cours	POST	/api/cours	Crée une nouvelle ressource (id généré par le serveur)
Remplacer un cours	PUT	/api/cours/{id}	Remplacement complet de la ressource
Modifier partiellement	PATCH	/api/cours/{id}	Mise à jour partielle (RFC 6902)
Supprimer un cours	DELETE	/api/cours/{id}	Supprime définitivement la ressource

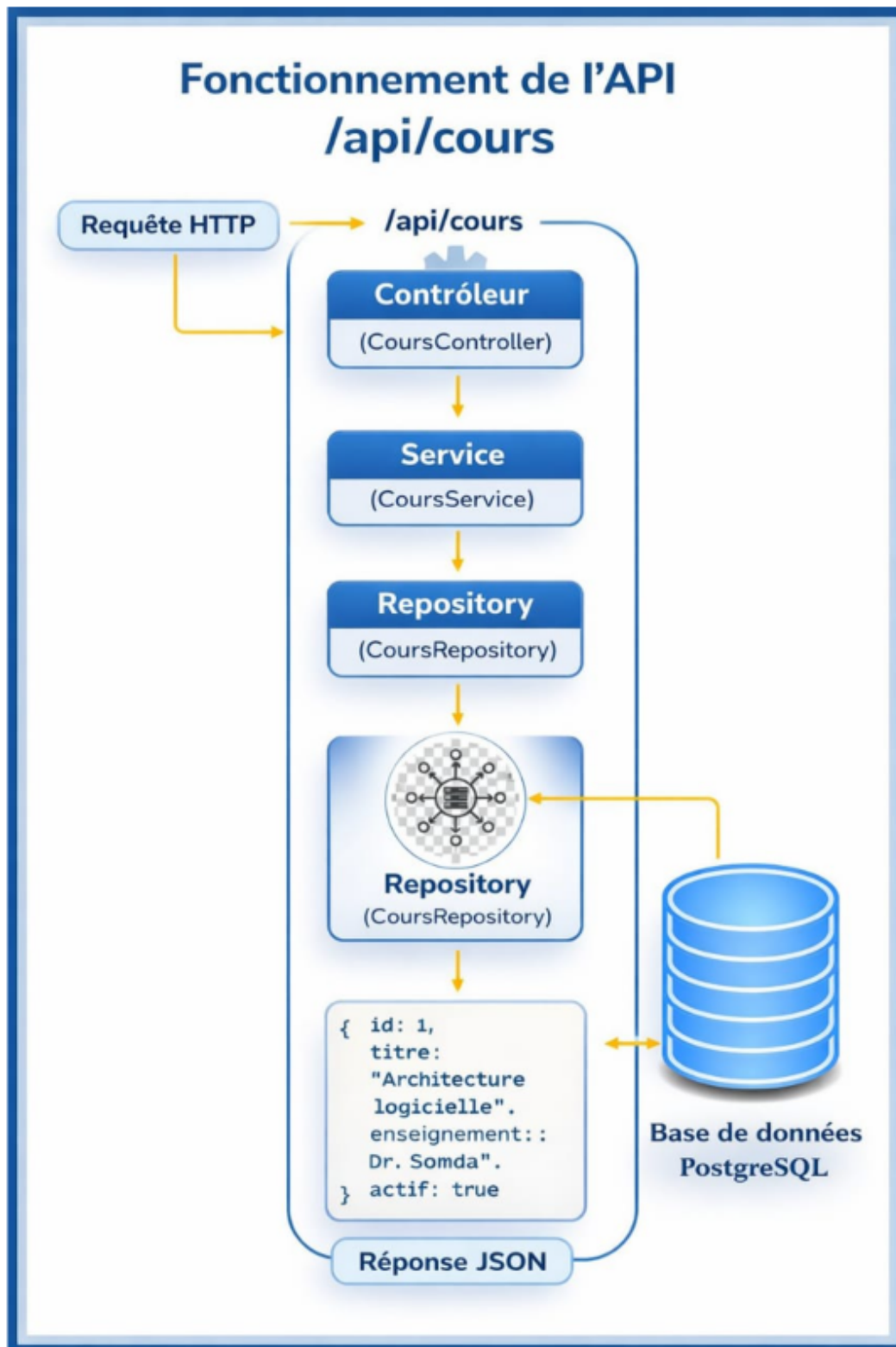


FIGURE 2.1 – Fonctionnement d'une API REST

Chapitre 3

Partie 2 – Consommation de l’API

Le sujet demande ensuite de tester l’API avec Postman ou curl, en commençant par une création de cours via **POST** puis une consultation via **GET**.

3.1 Création d’un cours (**POST**)

La création d’un cours se fait à travers une requête **POST** envoyée sur l’URI `/api/cours` avec un corps JSON. Le sujet fournit l’exemple suivant et indique que le code attendu est **201 Created**.

Listing 3.1 – Requête POST attendue

```
1 POST /api/cours
2 Content-Type: application/json
3
4 {
5   "titre": "Architecture Logicielle",
6   "enseignant": "Dr. Somda",
7   "volumeHoraire": 24,
8   "actif": true
9 }
```

Observation

Dans une API REST correcte, le serveur crée la ressource et renvoie un code **201 Created**. Selon l’implémentation, la réponse peut contenir la ressource créée avec son identifiant généré.

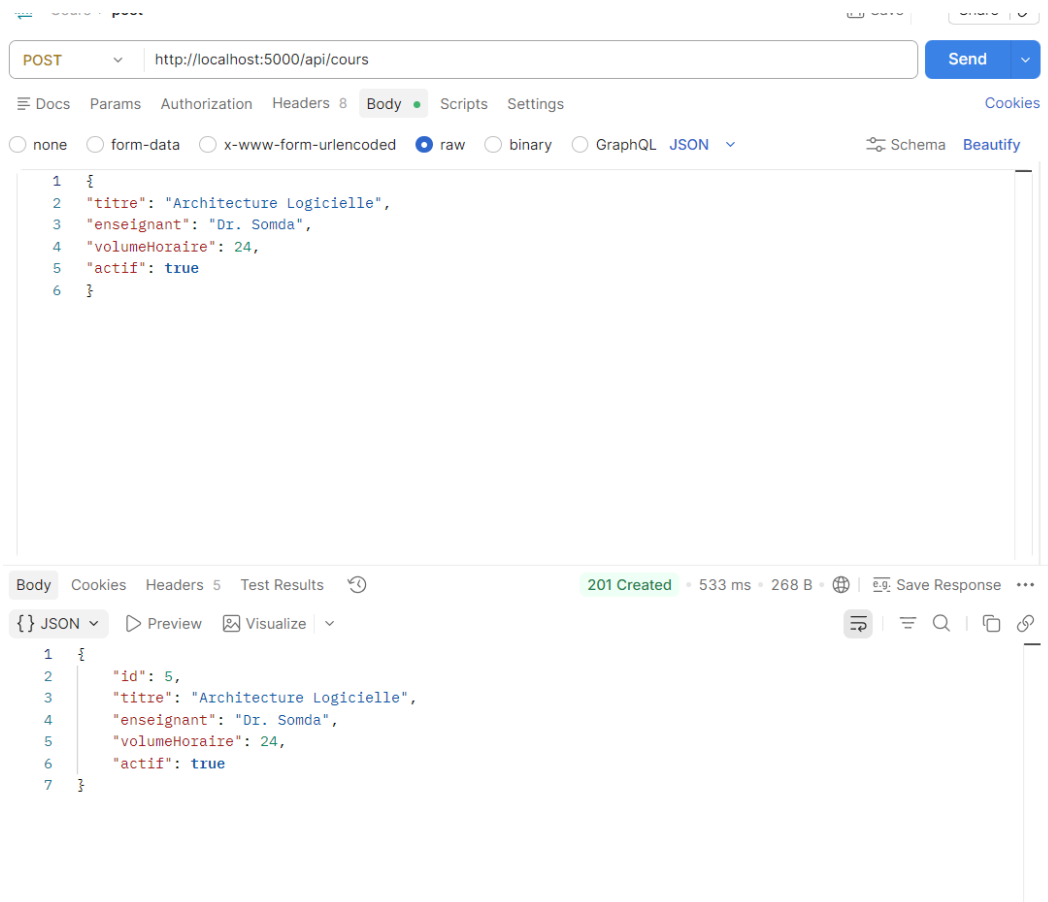


FIGURE 3.1 – Test POST

3.2 Consultation de la liste des cours (GET)

La consultation des cours se fait par une requête **GET** sur `/api/cours`. Le sujet précise que le code attendu est **200 OK**.

Listing 3.2 – Requête GET attendue

```
1 GET /api/cours
```

Observation

Une réponse correcte retourne un tableau JSON contenant l'ensemble des cours enregistrés.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:5000/api/cours
- Status:** 200 OK, 982 ms, 447 B
- Response Body (JSON):**

```
[{"id": 1, "titre": "Services Web", "enseignant": "Dr. Somda", "volumeHoraire": 40, "actif": false}, {"id": 4, "titre": "Genie logiciel", "enseignant": "Dr. Sabane", "volumeHoraire": 45, "actif": true}, {"id": 5, "titre": "Architecture Logicielle", "enseignant": "Dr. Somda", "volumeHoraire": 24, "actif": true}]
```

FIGURE 3.2 – Test GET

Chapitre 4

Partie 3 – Mise à jour PUT vs PATCH

Le sujet insiste sur la différence entre **PUT** et **PATCH**. Il fournit un exemple de remplacement complet avec **PUT** et un exemple de modification partielle avec **PATCH** en JSON Patch (RFC 6902).

4.1 Mise à jour complète avec **PUT**

La méthode **PUT** sert à remplacer complètement une ressource existante. Le sujet donne l'exemple suivant :

Listing 4.1 – Exemple de PUT

```
1 PUT /api/cours/1
2
3 {
4   "id": 1,
5   "titre": "Services Web Avances",
6   "enseignant": "Dr. Somda",
7   "volumeHoraire": 36,
8   "actif": true
9 }
```

Interprétation

Avec **PUT**, tous les champs de la ressource doivent être fournis. Il ne s'agit pas d'une mise à jour partielle, mais d'un remplacement complet.

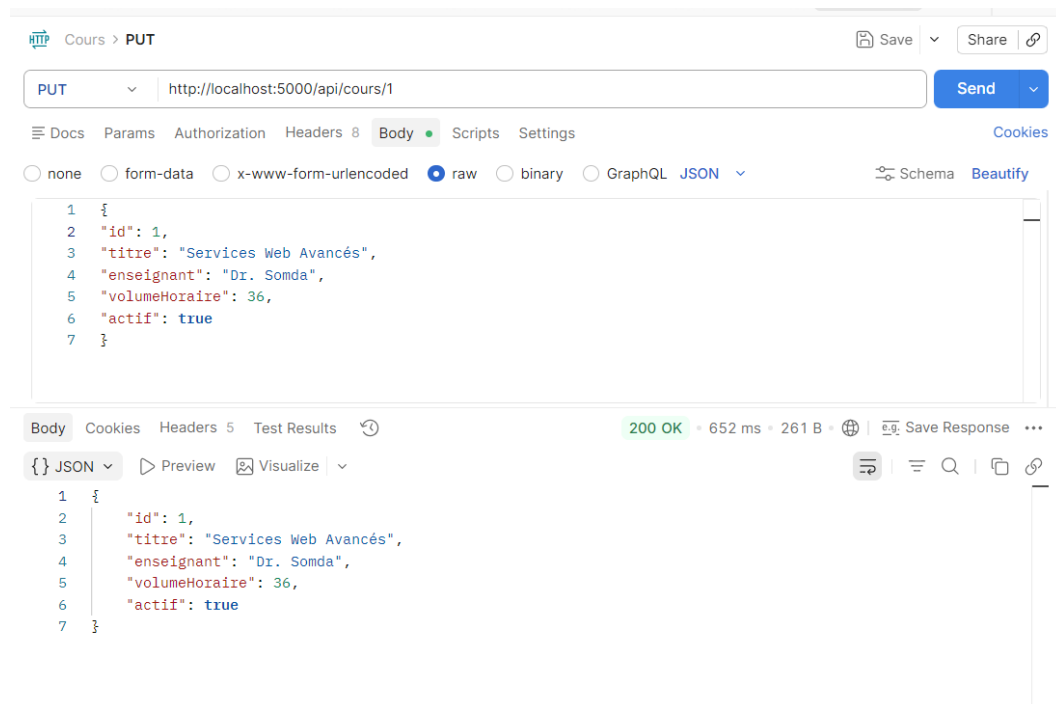


FIGURE 4.1 – Test PUT

4.2 Mise à jour partielle avec PATCH

La méthode **PATCH** permet de ne modifier qu'une partie de la ressource. Le sujet propose ici un format JSON Patch conforme à la RFC 6902.

Listing 4.2 – Exemple de PATCH

```
1 PATCH /api/cours/1
2 Content-Type: application/json-patch+json
3
4 [
5   { "op": "replace", "path": "/volumeHoraire", "value": 40 },
6   { "op": "replace", "path": "/actif", "value": false }
7 ]
```

Interprétation

Ici, seuls `volumeHoraire` et `actif` sont modifiés. Les autres champs ne changent pas.

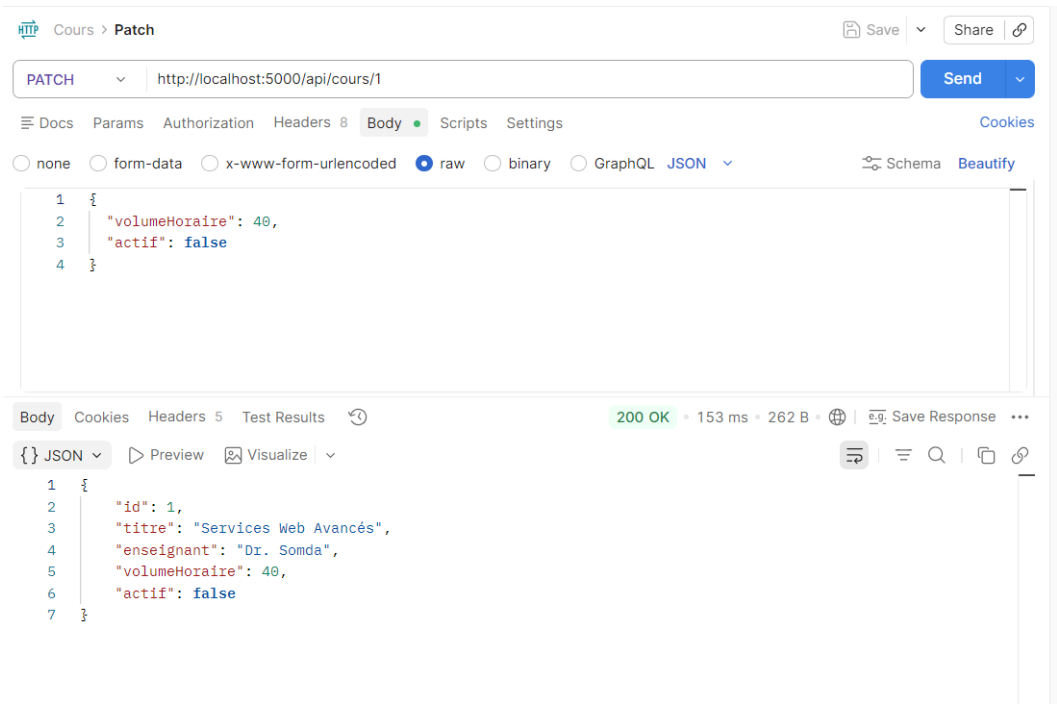


FIGURE 4.2 – Test PATCH

4.3 Comparaison entre PUT et PATCH

TABLE 4.1 – Comparaison détaillée PUT et PATCH

Critère	PUT	PATCH
Sémantique	Remplacement complet	Modification partielle
Idempotence	Oui	Pas nécessairement
Champs requis	Tous les champs	Seulement les champs modifiés
Format du corps	JSON complet	JSON Patch (RFC 6902)
Content-Type	application/json	application/json-patch+json
Risque principal	Écrasement accidentel de champs	Faible si bien implémenté
Cas d’usage typique	Formulaire de modification complète	Toggle, changement d’un seul champ

Chapitre 5

Partie 4 – Codes HTTP et erreurs

Le sujet demande de gérer plusieurs cas : ressource inexistante, JSON invalide, création réussie et suppression réussie. Il demande aussi de prévoir un message d'erreur JSON standard.

5.1 Catalogue des codes HTTP

TABLE 5.1 – Codes HTTP attendus selon la situation

Code	Libellé	Situation déclenchante
200	OK	GET , PUT ou PATCH réussi
201	Created	POST réussi — ressource créée
204	No Content	DELETE réussi — corps vide
400	Bad Request	Corps JSON invalide / syntaxe incorrecte
404	Not Found	Ressource introuvable (/api/cours/999)
422	Unprocessable	Syntaxe JSON valide mais contrainte métier violée
500	Internal Error	Erreur serveur non gérée

5.2 Format standard d'erreur

Le sujet propose le format JSON suivant :

Listing 5.1 – Message d'erreur JSON standard

```
1 {  
2   "error": "Cours non trouve",
```

```
3  "timestamp": "2026-02-20T10:30:00"
4  }
```

5.3 Erreur 404 – Ressource inexistante

Cette erreur apparaît lorsqu'on demande un cours qui n'existe pas, par exemple avec un identifiant absent de la base.

Listing 5.2 – Exemple de requête provoquant une 404

```
1 GET /api/cours/999
```

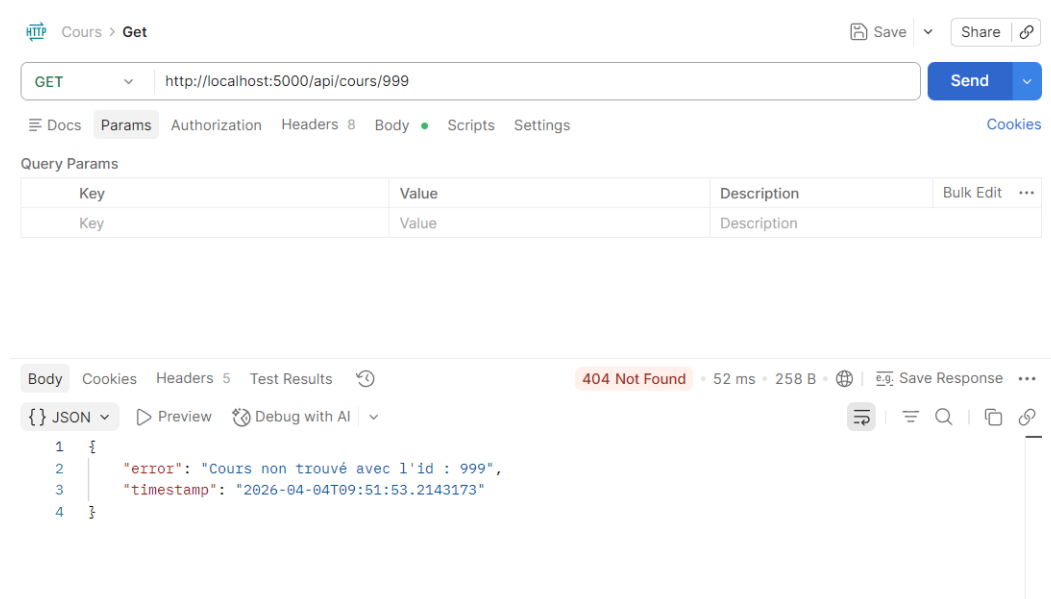


FIGURE 5.1 – Test error 404

5.4 Erreur 400 – JSON invalide

Cette erreur apparaît lorsqu'une requête contient un corps JSON mal formé ou non conforme.

Listing 5.3 – Exemple de requête provoquant une 400

```
1 POST /api/cours
2 Content-Type: application/json
3
4 {
5   "titre": "Architecture Logicielle",
6   "enseignant":
7 }
```

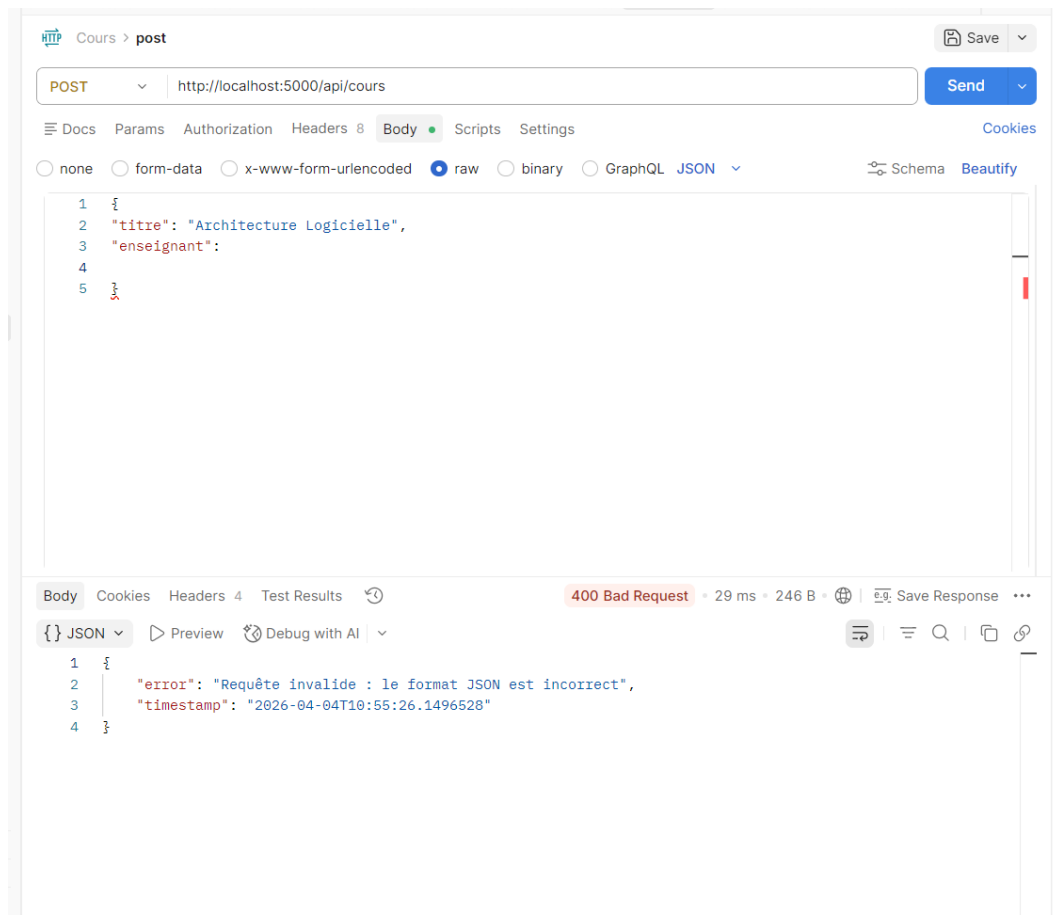


FIGURE 5.2 – Test error 400

5.5 Code 201 – Création réussie

Le code 201 confirme qu'une ressource a bien été créée.

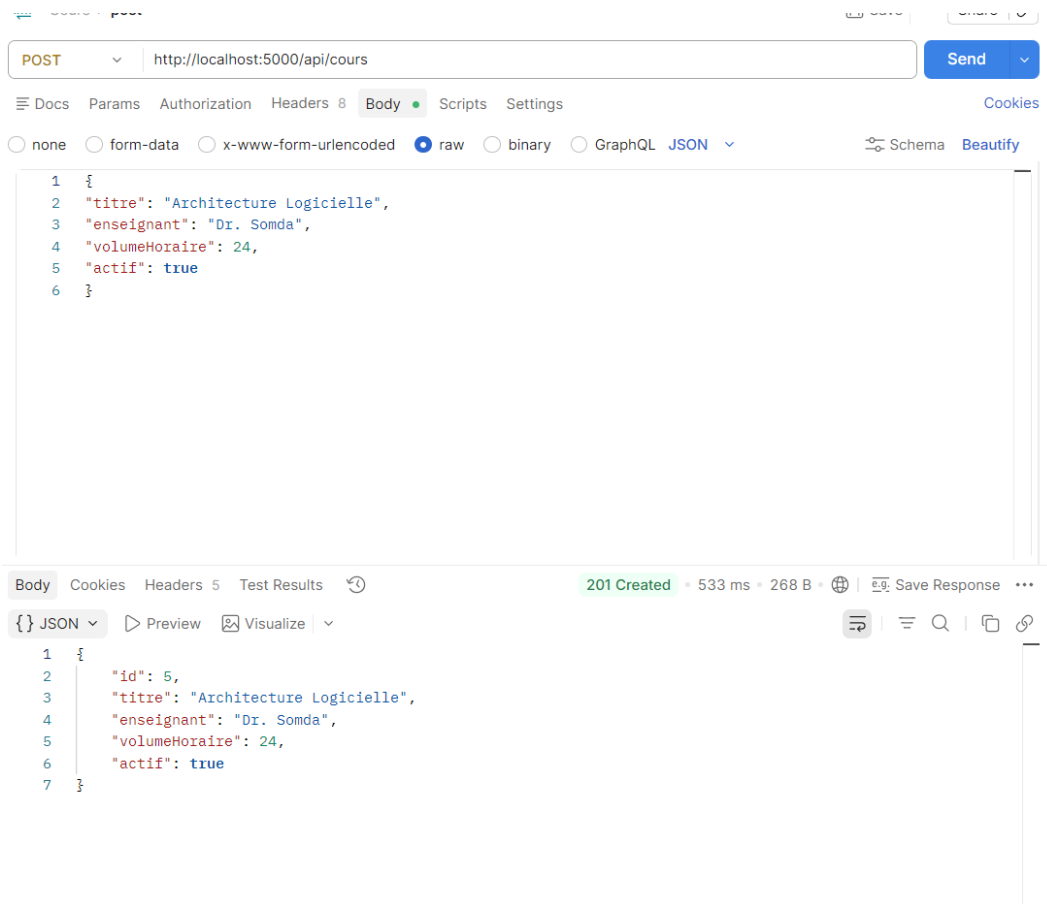


FIGURE 5.3 – Test creation reussie

5.6 Code 204 – Suppression réussie

Le code 204 confirme qu'une ressource a bien été supprimée sans contenu de réponse.

Listing 5.4 – Exemple de requête DELETE

```
1 DELETE /api/cours/1
```

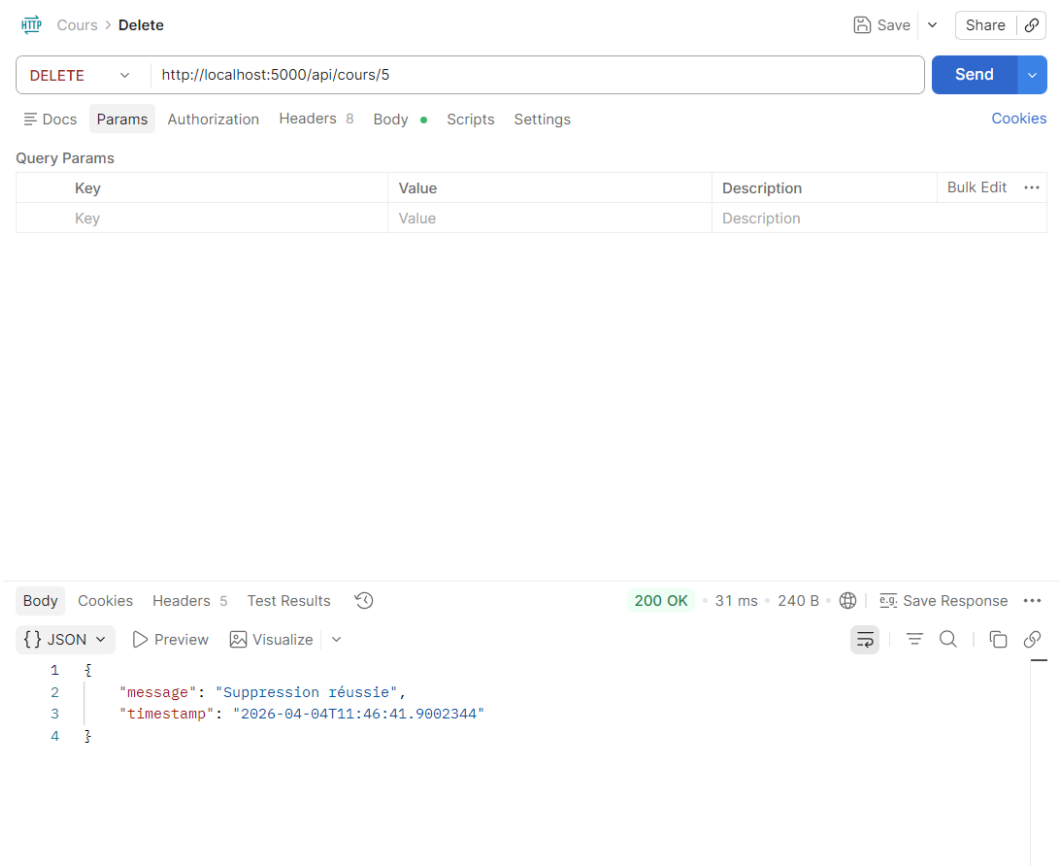



FIGURE 5.4 – Test suppression reussie

Chapitre 6

Partie 5 – Bonus (niveau Master)

Le sujet demande de choisir au moins une amélioration entre :

- une validation métier : `volumeHoraire > 0`;

6.1 Choix retenu : validation métier

Dans ce rapport, le bonus retenu est la validation métier imposant que `volumeHoraire` soit strictement supérieur à 0.

6.1.1 Justification

Un volume horaire négatif ou nul n'a pas de sens dans le contexte d'un cours universitaire. Cette contrainte permet donc de renforcer la cohérence métier des données.

6.1.2 Exemple de règle

Listing 6.1 – Exemple de contrainte métier

```
1 volumeHoraire > 0
```

6.1.3 Résultat attendu

Si la requête contient une valeur non valide, l'API doit refuser l'opération et renvoyer une erreur.

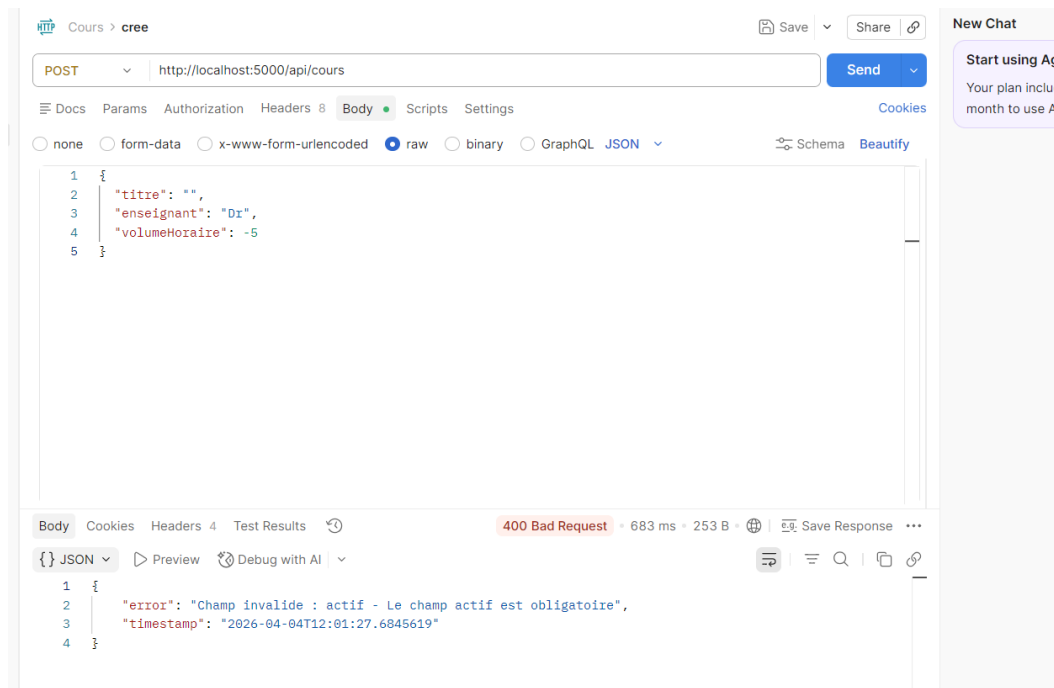


FIGURE 6.1 – Test champ obligatoire

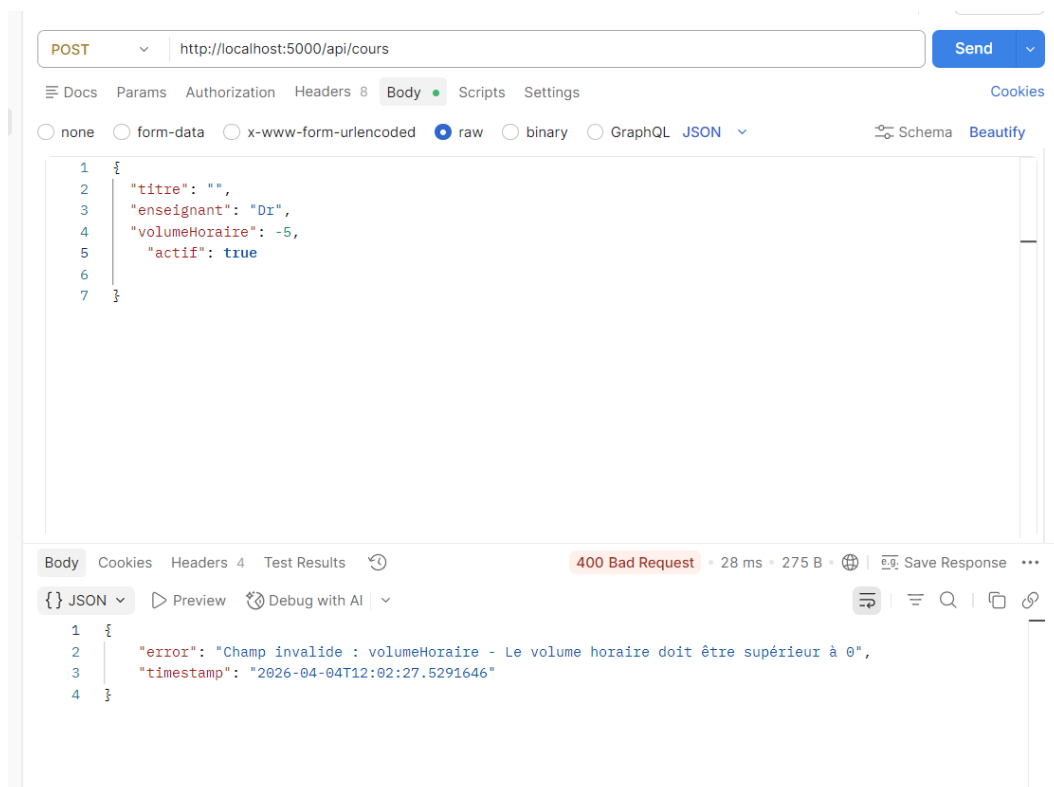


FIGURE 6.2 – Test volume horaire inferieur a 0

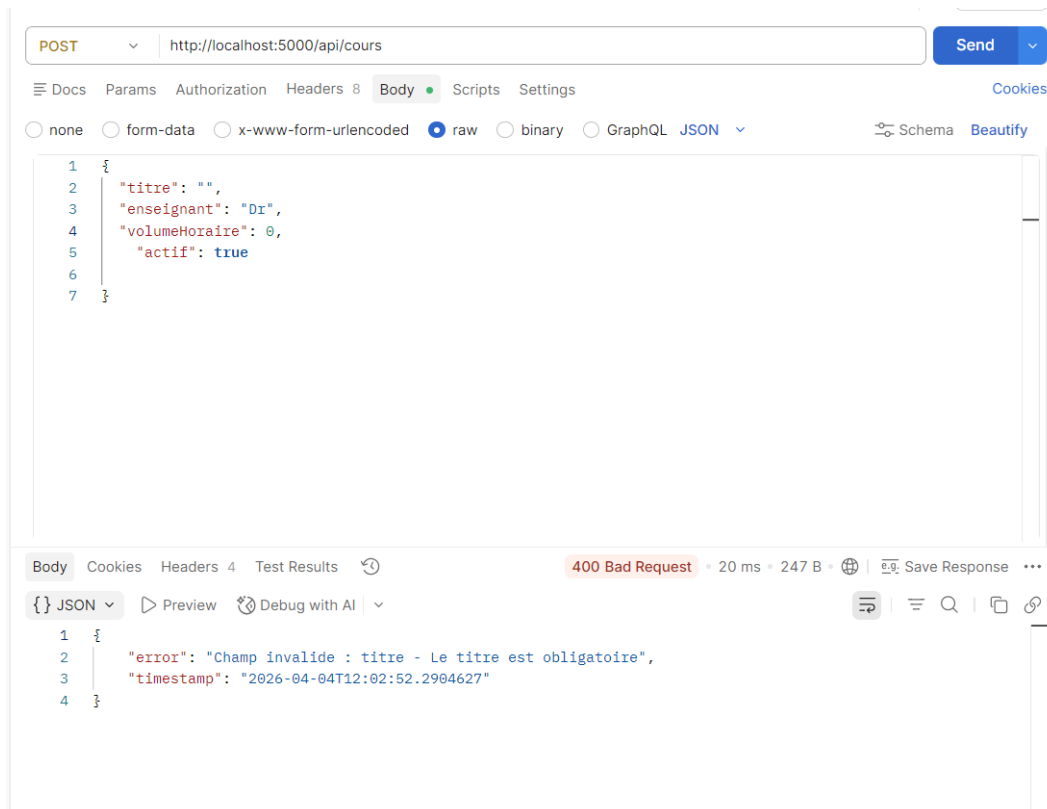


FIGURE 6.3 – Test volume horaire egal a 0

6.2 Alternative possible : pagination

Une autre amélioration possible consiste à ajouter une pagination sur la liste des cours. Cela permettrait de limiter le nombre d'éléments renvoyés dans une même réponse, par exemple :

Listing 6.2 – Exemple d'URL avec pagination

```
1 GET /api/cours?page=0&size=10
```

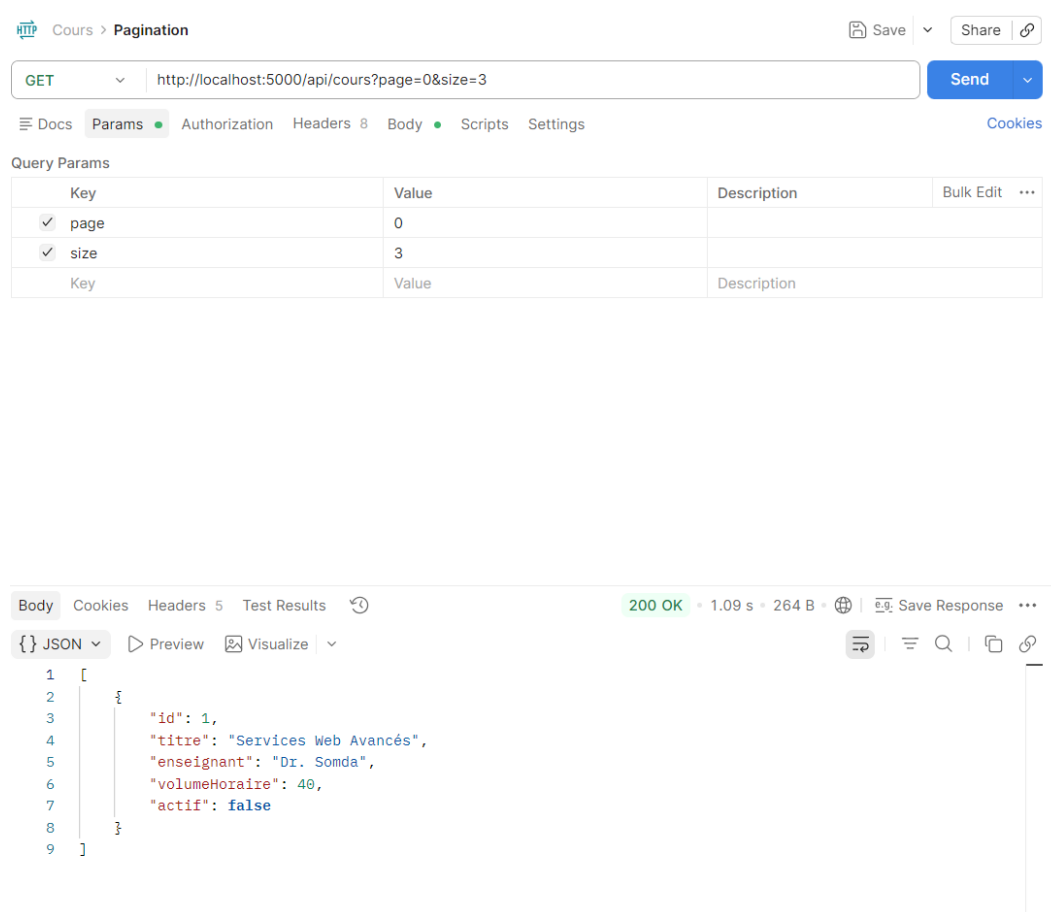


FIGURE 6.4 – Test pagination

Outils utilisés

Le sujet recommande l'usage de Postman, curl, Swagger / OpenAPI, Spring Boot

Dans le cadre de ce travail, les principaux outils mobilisés sont :

- **Postman** pour l'envoi et l'observation des requêtes HTTP ;
- **Spring Boot** ;
- **JAVA** langage de programmation ;
- **PostgreSQL** comme base de donnée ;

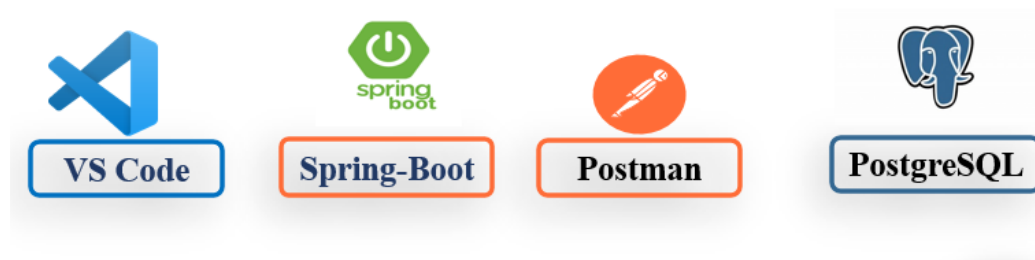


FIGURE 6.5 – Les outils

Conclusion

Ce travail pratique a permis de mettre en œuvre de manière concrète les principes fondamentaux des API REST dans un contexte applicatif simple mais représentatif. À travers la gestion d’une ressource *Cours*, nous avons pu concevoir une API respectant les bonnes pratiques en matière de structuration des URI, d’utilisation des méthodes HTTP et de manipulation des données au format JSON.

Les différentes opérations CRUD ont été implémentées et testées, permettant de vérifier le bon fonctionnement de l’API ainsi que la cohérence des réponses renvoyées. Une attention particulière a été portée à la gestion des erreurs, avec l’utilisation de codes HTTP adaptés et de messages explicites, améliorant ainsi la lisibilité et la compréhension côté client.

Par ailleurs, l’intégration de mécanismes de validation des données et de pagination constitue une amélioration significative, apportant une meilleure robustesse et une meilleure gestion des volumes de données.

Ce TP constitue ainsi une base solide pour le développement d’applications web modernes, et les notions acquises pourront être réutilisées dans des projets plus complexes intégrant des architectures distribuées.